Table of Contents

SOFTWARE DESIGN & ARCHITECTURE DOCUMENT

IntelliHire

AI-Powered Multilingual Voice Interview Preparation Platform

Document Type: SDD — Engineering Design Handbook

Version: 1.0 — Final (Consolidates Architecture v3–v6 + Enterprise Edition)

Audience: Engineering team beginning implementation

Companion document: IntelliHire Software Requirements Specification (SRS)

Chapter 1 — Executive Summary

1.1 What IntelliHire Is

IntelliHire is an AI-powered, multilingual, voice-based interview simulation and assessment platform. It conducts realistic, adaptive job interviews with candidates by voice, in their preferred language, using a resume-grounded question strategy, and produces a structured, explainable performance report at the end. The platform is delivered through three role-scoped panels — **Admin**, **Company**, and **Candidate** — so that a single deployment can serve platform operators, hiring organizations, and job candidates with appropriately scoped controls.

1.2 Problem Statement

Existing interview-preparation tools generally fall into one of two categories, both with structural limitations:

1. **Static question-bank tools** — a fixed list of questions per role/topic, delivered in a rigid order regardless of how the candidate is actually performing. These do not adapt difficulty, do not probe deeper on weak or interesting answers, and do not reflect the candidate's actual resume.
2. **Free-form "chat with an AI interviewer" tools** — flexible, but uncontrolled: the LLM decides everything (what to ask, when to stop, how hard to push), which makes the interview inconsistent, prone to topic drift, and impossible to audit or explain after the fact.

Neither approach gives a hiring organization a **controllable, explainable, and resume-grounded** interview process at scale.

1.3 Why Current Interview Platforms Fail

- They cannot explain *why* a question was asked or *why* a score was given, which limits recruiter trust.
- They rely on fixed question counts rather than actual topic coverage, producing interviews that are too short for complex candidates and too long for simple ones.
- They evaluate answers by transcript-guessing alone, without incorporating resume consistency, without protecting against prompt-injection attempts, and without any bot/script-reading detection.
- They give a hiring organization no configuration control — one-size-fits-all question style regardless of whether the hiring context is a campus drive or a senior technical round.

1.4 Objectives

- Conduct realistic, adaptive, resume-grounded voice interviews in multiple languages.

- Keep every AI decision (topic choice, difficulty change, follow-up, stopping point, scoring) **deterministic and auditable** wherever the decision doesn't require language understanding, and isolate genuine LLM judgment to a narrow, validated interface.
- Give companies **business-friendly configuration** (Interview Modes) without exposing internal AI mechanics.
- Make interview length **dynamic**, driven by actual resume complexity and live performance, not a fixed number.
- Validate every AI-generated question **before** it reaches a candidate.
- Make the platform's decisions **explainable** to both candidates and recruiters.
- Detect and neutralize prompt-injection and cheating attempts without disrupting the candidate experience.

1.5 Key Innovations

Innovation	Why it matters
Deterministic Interview Strategy Engine wrapped around a single LLM call per turn	Keeps cost and latency low while eliminating topic drift and unpredictable stopping behavior
Dynamic Interview Planning (resume-complexity-driven question budget)	Replaces fixed question counts with a defensible, resume-specific estimate
Company-facing Interview Modes over internal prompt strategies	Recruiters configure business intent, not AI implementation details
Deterministic Question Validator	Guarantees every question is on-topic, non-duplicate, and difficulty-aligned before the candidate ever hears it
AI Interview Explainability	Every AI decision in the interview is narrated from data the system already computed — no new LLM calls, full auditability
No permanent file storage	Resumes and audio are processed transiently; only derived structured data persists, minimizing data-retention risk

1.6 Business Value

For a hiring organization: consistent, scalable, resume-aware first-round screening that reduces recruiter time-per-candidate while producing an explainable, defensible report they can act on directly. For the platform operator: a configurable, multi-tenant product (via the Admin/Company hierarchy) that can serve many companies from one codebase without per-client forks.

1.7 Academic Value

The system demonstrates, as a coherent engineering artifact: LLM orchestration with strict deterministic guardrails, prompt-injection defense, multi-tenant RBAC design, dynamic algorithmic planning driven by

unstructured input (a resume), and AI explainability as a first-class system output rather than an afterthought — each a substantial, individually defensible topic, combined here into one working system.

1.8 Expected Outcomes

A production-shaped reference implementation covering: resume intelligence, adaptive multilingual voice interviewing, deterministic AI orchestration, a validated question pipeline, multi-tenant access control, recruiter-grade analytics and reporting, and transparent AI decision explanation — built entirely on a technology stack (FastAPI, MongoDB, Groq/Gemini, Sarvam AI, React) with no unnecessary infrastructure (no vector database, no object storage, no external cache), each omission justified in Chapter 19.

Chapter 2 — Complete Product Vision

2.1 Target Users

2.1.1 Admin

The platform operator. Owns the ceiling of what is possible on the platform: which languages, voices, LLM tiers, strategies, and Interview Modes exist and are enabled; which companies are active; platform-wide analytics and security oversight.

2.1.2 Company (Recruiter / Hiring Organization)

A tenant of the platform. Operates entirely within the ceiling Admin has set for them. Creates Interview Campaigns (a configured interview offering for a specific hiring need), selects an Interview Mode, voice, language(s), and whether candidates may make some choices themselves. Views only their own candidates' reports and analytics.

2.1.3 Candidate / Intern

The person being interviewed. Registers against a specific campaign (via an invite/link), uploads a resume, optionally makes delegated choices (language/domain, only if the company allowed it), takes the voice interview, and views their own report.

2.2 Complete User Journey

2.3 Business Workflow

1. Admin onboards a new company, setting the configuration ceiling appropriate to their contract tier.
2. Company sets up one or more campaigns per open role or hiring drive.
3. Candidates flow through campaigns; each interview is fully self-contained (resume → plan → interview → report), requiring no manual recruiter intervention mid-interview.
4. Recruiters review individual reports and the aggregate dashboard to make screening decisions, using the explainability section when they want to audit a specific score.

5. Admin monitors platform health, integrity signals (guardrail/anti-cheat triggers across all companies), and usage to inform product and account-management decisions.

2.4 How the Platform Operates (System-Level Summary)

IntelliHire is a stateless FastAPI application (all durable state in MongoDB) fronted by a React SPA with three role-based route trees. Every interview session's full lifecycle is driven by a single backend orchestrator (the Interview Manager) that delegates to a fixed set of deterministic and LLM-backed submodules — detailed component-by-component in Chapter 6. No component outside the LLM Service Layer ever calls an LLM provider directly, and no component other than the Question Validator gates what reaches the candidate.

Chapter 3 — Complete Functional Overview

This chapter briefly describes every feature; each is specified in exhaustive detail in its dedicated chapter (cross-referenced below) and in the companion SRS document's functional requirements (`FR-*` IDs).

Feature	Summary	Detailed in	SRS Requirements
Resume Parsing & Structuring	Extracts and structures resume content into a JSON profile; no permanent file storage	Ch. 13	FR-RES-1..6
Dynamic Interview Planning	Computes a resume-complexity-driven question budget, replacing fixed counts	Ch. 13	FR-PLAN-1..7
Blueprint Generation	Produces the ordered topic plan for the interview	Ch. 13	FR-BP-1..4
Interview Modes	Business-facing configuration of question style, thresholds, follow-up policy	Ch. 14	FR-MODE-1..9
Voice Interview Delivery	TTS/STT-driven, non-real-time-streaming voice interaction	Ch. 13	FR-VOICE-1..6
Question Validation	Deterministic pre-delivery validation of every AI-generated question	Ch. 15	FR-QV-1..11
Difficulty Control	Adaptive difficulty with cognitive-load-aware pacing	Ch. 13	FR-EVAL-4
Coverage Management	Live per-topic coverage/confidence tracking	Ch. 13	FR-BP-2
Completion Engine	Multi-lock dynamic stopping logic	Ch. 13	FR-COMP-1..5
Explainability	Deterministic narration of every AI decision	Ch. 18	FR-EXP-1..4
Reports	Structured, recruiter-quality, explainable final report	Ch. 17	FR-REP-1..4
Analytics	Company- and platform-level aggregate dashboards	Ch. 16	FR-AN-1..6
Guardrails	Deterministic prompt-injection detection	Ch. 6, 15	FR-SEC-1..2
Anti-Cheat	Deterministic script/pasted-text detection	Ch. 6	FR-SEC-3..4
Session Recovery	WebSocket-disconnect-tolerant session continuation	Ch. 6	FR-SESS-1..3
Administration	Platform-wide configuration and analytics	Ch. 4	FR-ADM-1..3

Chapter 4 — Three-Panel System

4.1 Admin Panel

4.1.1 Purpose

Gives the platform operator full control over what the platform can do and full visibility into how it is being used, without ever touching an individual company's or candidate's live interview.

4.1.2 Permissions

Admin tokens carry `role: "admin"` with no `company_id` / `campaign_id` scoping — Admin endpoints are the only ones permitted to query across all companies.

4.1.3 Pages & Navigation

```
Admin Panel
├── Dashboard                  (platform KPIs at a glance)
├── Companies
│   ├── Company List           (search, filter by status)
│   ├── Company Detail         (allowed_* ceiling editor, usage summary)
│   └── New Company             (onboarding form)
├── Platform Settings
│   ├── Voice Management        (enable/disable Sarvam voices platform-wide)
│   ├── Language Management      (enable/disable languages platform-wide)
│   ├── Interview Mode Management (enable/disable/define modes)
│   └── Strategy Configuration    (enable/disable internal strategies — visible to Admin
only, never to Company)
├── Analytics                   (platform-wide, unscoped version of Company Analytics —
Ch. 16)
├── Security Logs                (guardrail triggers, anti-cheat flags, across all
companies)
├── User Management              (admin/company user accounts, password resets,
suspensions)
└── System Health                (LLM provider status, Sarvam status, error rates, queue
depth)
```

4.1.4 Key Workflows

- **Onboard a company**: Admin fills the New Company form (name, contact email, initial `allowed_*` ceiling) → `POST /api/admin/companies` → company account provisioned, credentials issued.
- **Adjust a company's ceiling**: Admin edits `allowed_languages` / `allowed_voices` / `allowed_strategies` / `allowed_interview_modes` / `allowed_llm_tiers` on the Company Detail page → `PATCH /api/admin/companies/{id}` → takes effect immediately for any campaign the company creates or edits afterward (does not retroactively change already-running interviews).

- **Enable a new Interview Mode**: Admin adds/edits a `strategy_definitions` / `interview_mode_definitions` document via the Interview Mode Management page → available to companies whose ceiling includes it, with no backend deployment.
- **Review integrity incidents**: Admin filters Security Logs by company/date/incident type to identify patterns (e.g., a spike in guardrail triggers from one company's campaign, which may indicate candidates are being coached to attempt prompt injection).

4.2 Company Panel

4.2.1 Purpose

Lets a hiring organization configure and run interview campaigns and review results, entirely within the ceiling Admin has granted them.

4.2.2 Pages & Navigation

```
Company Panel
├── Dashboard                  (this company's KPIs — Ch. 16)
├── Campaigns
│   ├── Campaign List           (active/closed)
│   ├── Campaign Detail          (config summary, candidate list, per-campaign
analytics)
│   └── New Campaign              (role, Interview Mode, voice/language, delegation
flags)
├── Candidates
│   ├── Candidate List             (across all campaigns, filterable)
│   └── Candidate Report View       (full report incl. Explainability)
├── Analytics                       (Ch. 16, full detail)
├── Exports                          (CSV/PDF downloads, historical export list)
├── Profile                           (company details)
└── Settings                           (branding for PDF reports — logo, accent color)
```

4.2.3 Company Lifecycle

`Admin onboards company → Company logs in → Company sets branding (optional) → Company creates first campaign → Company remains active until Admin suspends or company account is closed.`

4.2.4 Campaign Lifecycle

`Company creates campaign (status: active) → candidates register and interview against it → Company may close the campaign (status: closed) once hiring need is filled → closed campaigns remain fully visible in Reports/Analytics but no longer accept new candidate registrations.`

4.2.5 Key Workflows

- **Create a campaign**: Company fills the New Campaign form, choosing only from Admin-permitted options (Interview Mode dropdown shows only `allowed_interview_modes`; voice/language pickers

show only `allowed_voices` / `allowed_languages`) → `POST /api/company/campaigns` → campaign link generated for distribution to candidates.

- **Review a candidate**: Company opens Candidate Report View → sees full scored report, Resume Match Analysis, Interview Risk Assessment, and the collapsible Explainability section → may export as PDF.
- **Review aggregate performance**: Company opens Analytics, filters by campaign and time range, reviews weak/strong skill trends and integrity metrics across all their candidates.

4.3 Candidate/Intern Panel

4.3.1 Pages & Navigation

```
Candidate Panel
├── Registration              (via campaign-scoped invite link)
├── Profile                    (name, contact — minimal, no resume file retained)
├── Resume Upload               (upload, see extracted structured profile for
confirmation)
├── Interview Setup               (language/domain choice, ONLY if delegated by the
company)
├── Interview Screen
│   ├── Question Display/Audio     (TTS playback)
│   ├── Voice Controls               (Start Answering / Submit Answer, recording
indicator)
│   └── Progress Indicator             (topic coverage progress, non-revealing of scores
mid-interview)
├── Interview History               (past interviews, if the candidate has taken more
than one)
├── Report                           (own report, incl. Explainability and
Recommendations)
└── Settings                          (language preference, notification preference)
```

4.3.2 Key Workflows

- **Take an interview**: Candidate follows a campaign invite link → registers → uploads resume → (optionally) selects language/domain → interview begins automatically once the Blueprint and Question Budget are ready → proceeds turn-by-turn until the Completion Engine ends it → report becomes available.
- **Review results**: Candidate opens Report → sees scores, strengths/weaknesses, improvement plan, and — distinctly — the Explainability section explaining why the interview unfolded as it did.

Chapter 5 — Complete System Workflow

5.1 End-to-End Flow


User Registration
Campaign Association (candidate registers against a specific company campaign)
Resume Upload → Parse → Clean → LLM Structuring → Structured Profile (MongoDB)
(original file discarded)
Experience/Skill/Project Complexity Analysis → Dynamic Question Budget
Blueprint Generation (LLM) → Interview Mode Manager → Strategy Selector
Per-Turn Loop
TTS (question) → Candidate listens → Candidate speaks → STT
Guardrail Router (exploit check) → LLM (evaluate + next question)
Question Validator (duplicate/relevance/difficulty/clarity checks)
pass
fail → regenerate (bounded retries)
Difficulty Controller (+ Adaptive Pace Adjuster) → Coverage Manager
Follow-up Decision Engine → Completion Engine (multi-lock check)
(loop continues until Completion Engine fires)
Aggregation (Python) → LLM (report sections) → Explainability Generator (deterministic)
interview_reports (scores + explainability)
Candidate Report View
Company Report View
Company Analytics Dashboard
(aggregated across all sessions)
Admin Platform-Wide Analytics

5.2 Sequence Diagram — Full Interview Lifecycle


Candidate
Frontend
FastAPI
Sarvam
LLM Service
Strategy Engine
Validator
MongoDB
register
POST /candidates
upload resume
POST /resume/upload
parse/clean
structure_resume
resume_profile
save
start interview
POST /interview/start
complexity analysis + question budget
generate_blueprint
blueprint
save
(loop begins — one iteration shown)
bulbul:v3
audio
listens, answers
POST /answer
saaras:v3
transcript
guardrail check (in-process)
evaluate_and_generate
evaluation + question
validate


pass/fail
update coverage, difficulty, follow-up/completion check
save turn
(repeat loop, or fall through to completion)
— completion triggered —
generate_final_report
report sections
Explainability Generator (local, deterministic)
save report
WS: complete
view report
GET /interview/{id}/report
fetch
report

5.3 Activity Diagram — Per-Turn Decision Logic

Chapter 6 — Architecture (Complete Backend Component Reference)

Every backend component is documented with: Purpose, Responsibilities, Key Methods, Data Flow, Dependencies, Inputs, Outputs. Components are presented in pipeline order.

6.1 Interview Manager

Aspect	Detail
Purpose	Sole orchestrator of a session's lifecycle; owns phase transitions and delegates every specialized decision to the correct submodule.
Responsibilities	Session start/stop, per-turn coordination, MongoDB reads/writes for session state, invoking Sarvam client, invoking Guardrail Router, invoking the Interview Strategy Engine and Question Validator in the correct order.
Key Methods	`start_session()`, `get_next_question_audio()`, `process_answer()`, `finalize_session()`
Data Flow	Receives candidate audio/events from the API layer → orchestrates STT → Guardrail → LLM Service → Validator → Strategy Engine submodules → persists turn → returns next question audio reference.
Dependencies	LLM Service, Sarvam Client, Interview Strategy Engine, Question Validator, Difficulty Controller, Resume Consistency Checker, MongoDB
Inputs	`session_id`, audio bytes, response timing
Outputs	`TurnResult` (evaluation, next question, or finalized report)

6.2 Dynamic Interview Planner

Aspect	Detail
Purpose	Replaces fixed question counts with a resume-complexity-driven question budget.
Responsibilities	Runs Experience Analyzer, Skill Complexity Analyzer, Project Complexity Analyzer; combines their scores into a weighted question-budget formula; enforces sanity floor/ceiling.
Key Methods	`analyze_experience()`, `analyze_skill_complexity()`, `analyze_project_complexity()`, `compute_question_budget()`
Data Flow	Runs once, immediately after resume structuring, before Blueprint Generation.
Dependencies	Resume Profile (from LLM Service's structuring output); no LLM call of its own — purely numeric scoring over already-structured fields.
Inputs	`resume_profile`, `interview_type`, `interview_mode`, `role`

Aspect	Detail
Outputs	`{min_questions, max_questions}`, complexity scores (persisted to Interview State)

6.3 Blueprint Generator

Aspect	Detail
Purpose	Produces the ordered, weighted topic plan for the interview.
Responsibilities	One LLM Service call, informed by resume profile + question budget + interview type/role; initializes the Topic Coverage Map in lockstep.
Key Methods	`generate_blueprint()`
Dependencies	LLM Service
Inputs	`resume_profile`, `interview_type`, `role`, question budget
Outputs	`list[TopicPlan]`

6.4 Interview Mode Manager

Aspect	Detail
Purpose	Translates a company-facing Interview Mode selection into internal strategy and rule parameters.
Responsibilities	Reads `campaign.question_strategy` (business label) → resolves internal strategy ID, follow-up ceiling, confidence/saturation thresholds, difficulty policy, and (for HR mode) behavioral template set.
Key Methods	`resolve_mode(mode_id) -> ModeConfig`
Dependencies	`interview_mode_definitions` collection (data-driven, not hardcoded)
Inputs	`mode_id`
Outputs	`ModeConfig` object consumed by the Strategy Selector, Difficulty Controller, and Completion Engine

6.5 Strategy Selector / Prompt Builder

Aspect	Detail
Purpose	Builds the strategy-specific context block (Strategy A token-driven, Strategy B conversational bridging, or Hybrid) merged into the evaluation/question-generation prompt.
Responsibilities	Pure function — no LLM call itself; assembles the structured instruction block the LLM Service will interpolate into its template.

Aspect	Detail
Key Methods	`build_context(strategy_id, topic_plan, difficulty, state, was_follow_up, topic_changed)`
Dependencies	None external — reads only from Interview State and the resolved `ModeConfig`.
Inputs	Strategy ID, current topic/difficulty/state
Outputs	`dict` context block

6.6 LLM Service Layer

Aspect	Detail
Purpose	Single, provider-independent entry point for every LLM call in the system.
Responsibilities	Prompt template management, provider selection (Groq primary, Gemini fallback), retries with backoff, strict JSON schema validation of every response.
Key Methods	`structure_resume()`, `generate_blueprint()`, `evaluate_and_generate()`, `check_resume_consistency()`, `generate_final_report()`
Dependencies	`GroqProvider`, `GeminiProvider` (both implementing a common `LLMProvider` protocol)
Inputs	Task-specific structured inputs (resume text, Interview State, etc.)
Outputs	Validated Pydantic objects — never raw provider text

6.7 Question Validator

Aspect	Detail
Purpose	Deterministic gate between LLM question generation and the candidate.
Responsibilities	Duplicate detection (keyword + embedding similarity), topic/blueprint relevance, difficulty alignment, follow-up validity, clarity/length, Interview Mode compliance.
Key Methods	`validate(question, context) -> ValidationResult`
Dependencies	A lightweight sentence-embedding model (local, not an LLM call) for semantic similarity
Inputs	Candidate question text, session context (already-asked questions, topic, difficulty, mode)
Outputs	Pass/fail + failed-rule list; logged regardless of outcome

6.8 Difficulty Controller (incl. Adaptive Pace Adjuster)

Aspect	Detail
Purpose	Deterministically adjusts difficulty turn-by-turn.

Aspect	Detail
Responsibilities	Score-based tier movement; `CALIBRATE_HOLD` flag when a high score is paired with unusually long response time (cognitive-load protection).
Key Methods	`update_difficulty(state, latest_score, response_time, word_count)`
Dependencies	None external
Inputs	Latest evaluation score, response timing/length
Outputs	Updated difficulty tier (or hold flag)

6.9 Coverage Manager

Aspect	Detail
Purpose	Single source of truth for topic progress.
Responsibilities	Updates per-topic status, confidence (recency-weighted), saturation detection, failure-pivot detection; recomputes aggregate coverage/confidence.
Key Methods	`update_coverage()`, `check_topic_saturation()`, `check_failure_pivot()`
Inputs	Topic name, latest score, follow-up flag
Outputs	Updated `TopicCoverage` entries + aggregate percentages

6.10 Follow-up Decision Engine

Aspect	Detail
Purpose	Decides per-turn whether to probe deeper or move on.
Responsibilities	Combines coverage state with the LLM's `suggests_follow_up` signal and the active Interview Mode's follow-up policy.
Key Methods	`needs_follow_up(state, evaluation, topic_plan)`
Inputs	Evaluation result, topic coverage, mode config
Outputs	Boolean decision

6.11 Completion Engine

Aspect	Detail
Purpose	Multi-lock dynamic stopping logic.
Responsibilities	Checks hard resource constraint (dynamic max questions) and qualitative saturation (mandatory topics covered + confidence threshold); triggers Emergency Exit or Clean Completion.

Aspect	Detail
Key Methods	`should_complete_interview(state) -> (bool, reason)`
Inputs	Interview State
Outputs	Completion decision + reason (feeds directly into Explainability)

6.12 Resume Consistency Checker

Aspect	Detail
Purpose	Flags mismatches between resume claims and demonstrated performance.
Responsibilities	Per-turn lightweight check (part of the evaluation call) + end-of-interview comprehensive LLM Service call.
Key Methods	`check_resume_consistency(resume_profile, turns)`
Inputs	Resume profile, all turns
Outputs	`ConsistencyReport` feeding into Interview Risk Assessment

6.13 Guardrail Router

Aspect	Detail
Purpose	Deterministic prompt-injection/exploit detection.
Responsibilities	Pattern-matches every submitted transcript before it reaches evaluation; on detection, skips evaluation and serves a neutral redirect.
Key Methods	`check_guardrail(transcript) -> bool`
Inputs	Raw transcript
Outputs	Boolean + side-effect (exploitation counter increment)

6.14 Anti-Cheat Detection

Aspect	Detail
Purpose	Deterministic detection of likely script-reading/pasted-AI-text answers.
Responsibilities	Words-per-minute + structural-pattern heuristics on the transcript.
Key Methods	`detect_cheating_risk(transcript, response_time)`
Inputs	Transcript, response time
Outputs	Boolean flag, recorded on the turn, recruiter-visible only

6.15 Recovery Manager

Aspect	Detail
Purpose	WebSocket-disconnect-tolerant session continuation.
Responsibilities	On disconnect, marks `last_disconnected_at`; on reconnect within the recovery window, replays the last known question from `interview_state` without any new LLM call.
Key Methods	Embedded in the `/ws/interview/{session_id}` handler
Inputs	Session ID, JWT
Outputs	Resumed WebSocket stream

6.16 Report Generator

Aspect	Detail
Purpose	Assembles the final structured report.
Responsibilities	Precomputes all quantitative aggregates in Python; makes one LLM Service call for qualitative sections; assembles the full `interview_reports` document.
Key Methods	`generate_report(session)`
Inputs	Full session document, resume profile
Outputs	`interview_reports` document

6.17 Explainability Generator

Aspect	Detail
Purpose	Deterministically narrates every AI decision made during the interview.
Responsibilities	Template-fills explanations from already-stored data (blueprint tags, score history, `follow_up_reason`, Completion Engine's trigger reason, Readiness Score formula) — never calls the LLM.
Key Methods	`generate(session)`
Inputs	Full session document
Outputs	`explainability` object, nested in `interview_reports`

6.18 Analytics Service

Aspect	Detail
Purpose	Computes aggregate metrics for the Company and Admin dashboards.
Responsibilities	Runs MongoDB aggregation pipelines scoped by `company_id` / `campaign_id` /time range; no data duplication — computed on demand or cached briefly, never a second source of truth.
Key Methods	`get_dashboard_metrics(scope, filters)`
Inputs	Scope (company/platform), filters (campaign, date range)
Outputs	Metrics payload consumed by the frontend dashboard components

Chapter 7 — Complete Folder Structure

```
intellihire/
├── frontend/
│   ├── public/
│   ├── src/
│   │   ├── main.jsx
│   │   ├── App.jsx
│   │   ├── routes/
│   │   │   ├── AdminRoutes.jsx
│   │   │   ├── CompanyRoutes.jsx
│   │   │   ├── CandidateRoutes.jsx
│   │   │   └── ProtectedRoute.jsx          # role-gate wrapper
│   │   ├── layouts/
│   │   │   ├── AdminLayout.jsx
│   │   │   ├── CompanyLayout.jsx
│   │   │   └── CandidateLayout.jsx
│   │   ├── pages/
│   │   │   ├── admin/
│   │   │   │   ├── Dashboard.jsx
│   │   │   │   ├── Companies.jsx
│   │   │   │   ├── CompanyDetail.jsx
│   │   │   │   ├── PlatformSettings.jsx
│   │   │   │   ├── Analytics.jsx
│   │   │   │   ├── SecurityLogs.jsx
│   │   │   │   └── SystemHealth.jsx
│   │   │   ├── company/
│   │   │   │   ├── Dashboard.jsx
│   │   │   │   ├── Campaigns.jsx
│   │   │   │   ├── CampaignDetail.jsx
│   │   │   │   ├── NewCampaign.jsx
│   │   │   │   ├── Candidates.jsx
│   │   │   │   ├── CandidateReport.jsx
│   │   │   │   ├── Analytics.jsx
│   │   │   │   └── Settings.jsx
│   │   │   └── candidate/
│   │   │       ├── Registration.jsx
│   │   │       ├── ResumeUpload.jsx
│   │   │       ├── InterviewSetup.jsx
│   │   │       ├── InterviewRoom.jsx
│   │   │       ├── InterviewHistory.jsx
│   │   │       └── ReportView.jsx
│   │   ├── components/
│   │   │   ├── common/
│   │   │   │   ├── Button.jsx
│   │   │   │   ├── Card.jsx
│   │   │   │   ├── Modal.jsx
│   │   │   │   ├── Skeleton.jsx
│   │   │   │   └── Badge.jsx
│   │   │   ├── charts/
│   │   │   │   ├── ScoreLineChart.jsx        # Recharts wrapper
│   │   │   │   ├── SkillBarChart.jsx
│   │   │   │   ├── DifficultyPieChart.jsx
│   │   │   │   └── CounterCard.jsx           # Framer Motion animated counter
│   │   │   ├── interview/
│   │   │   │   ├── AudioRecorder.jsx
│   │   │   │   ├── AudioPlayer.jsx
│   │   │   │   ├── QuestionCard.jsx
│   │   │   │   ├── DifficultyIndicator.jsx
│   │   │   │   └── ProgressBar.jsx
```

```
│   │   │   └── report/
│   │   │       ├── ScoreRing.jsx
│   │   │       ├── RiskAssessmentPanel.jsx
│   │   │       ├── ExplainabilitySection.jsx
│   │   │       └── RecommendationsList.jsx
│   │   ├── hooks/
│   │   │   ├── useInterviewSocket.js
│   │   │   ├── useAudioRecorder.js
│   │   │   ├── useAuth.js
│   │   │   └── useAnalyticsFilters.js
│   │   ├── context/
│   │   │   ├── AuthContext.jsx
│   │   │   └── ThemeContext.jsx
│   │   ├── api/
│   │   │   ├── client.js                    # axios/fetch wrapper, JWT injection
│   │   │   ├── candidates.js
│   │   │   ├── companies.js
│   │   │   ├── interview.js
│   │   │   └── analytics.js
│   │   └── utils/
│   │       ├── formatters.js
│   │       └── validation.js
│   ├── package.json
│   └── tailwind.config.js
│
├── backend/
│   ├── app/
│   │   ├── main.py                          # FastAPI app init, router mounting
│   │   ├── api/
│   │   │   ├── admin/
│   │   │   │   ├── companies.py
│   │   │   │   ├── strategies.py
│   │   │   │   ├── interview_modes.py
│   │   │   │   └── analytics.py
│   │   │   ├── company/
│   │   │   │   ├── campaigns.py
│   │   │   │   ├── candidates.py
│   │   │   │   ├── reports.py
│   │   │   │   └── analytics.py
│   │   │   ├── candidates.py
│   │   │   ├── resume.py
│   │   │   ├── interview.py
│   │   │   └── ws.py
│   │   ├── core/
│   │   │   ├── interview_manager.py
│   │   │   ├── difficulty_controller.py
│   │   │   ├── consistency_checker.py
│   │   │   ├── interview_state.py
│   │   │   ├── explainability/
│   │   │   │   └── explainability_generator.py
│   │   │   └── strategy/
│   │   │       ├── strategy_engine.py
│   │   │       ├── interview_mode_manager.py
│   │   │       ├── dynamic_interview_planner.py
│   │   │       ├── experience_analyzer.py
│   │   │       ├── complexity_analyzer.py
│   │   │       ├── blueprint_generator.py
│   │   │       ├── topic_priority.py
│   │   │       ├── coverage_manager.py
```

```
│   │   │       ├── follow_up_engine.py
│   │   │       ├── completion_engine.py
│   │   │       ├── question_budget.py
│   │   │       ├── question_guard.py
│   │   │       └── guardrail.py
│   │   ├── llm/
│   │   │   ├── llm_service.py
│   │   │   ├── providers/
│   │   │   │   ├── base.py
│   │   │   │   ├── groq_provider.py
│   │   │   │   └── gemini_provider.py
│   │   │   └── prompts/
│   │   │       ├── resume_structuring.py
│   │   │       ├── evaluation_and_question.py
│   │   │       ├── consistency_check.py
│   │   │       ├── final_report.py
│   │   │       ├── blueprint_generation.py
│   │   │       ├── strategy_a_wording.py
│   │   │       ├── strategy_b_bridging.py
│   │   │       ├── strategy_hybrid.py
│   │   │       └── strategy_selector.py
│   │   ├── validation/
│   │   │   └── question_validator.py
│   │   ├── analytics/
│   │   │   ├── dashboard_service.py
│   │   │   ├── metrics.py
│   │   │   └── aggregation.py
│   │   ├── reports/
│   │   │   ├── report_generator.py
│   │   │   ├── pdf_renderer.py                # WeasyPrint pipeline
│   │   │   └── templates/
│   │   │       ├── report_template.html
│   │   │       └── dashboard_export_template.html
│   │   ├── speech/
│   │   │   └── sarvam_client.py
│   │   ├── resume_processing/
│   │   │   ├── parser.py
│   │   │   └── cleaner.py
│   │   ├── rbac/
│   │   │   ├── permissions.py
│   │   │   └── models.py
│   │   ├── db/
│   │   │   ├── mongo.py
│   │   │   ├── models.py
│   │   │   └── indexes.py
│   │   ├── auth/
│   │   │   └── jwt_handler.py
│   │   ├── middleware/
│   │   │   ├── logging_middleware.py
│   │   │   └── error_handler.py
│   │   └── config/
│   │       └── settings.py                    # env var loading
│   ├── tests/
│   │   ├── unit/
│   │   ├── integration/
│   │   └── api/
│   ├── requirements.txt
│   └── Dockerfile
│
```

```
├── docs/
│   ├── SRS.pdf
│   ├── SDD.pdf
│   └── api_reference.md
│
├── scripts/
│   ├── seed_dev_data.py
│   └── run_migrations.py
│
├── docker-compose.yml
└── .env.example
```

Chapter 8 — Frontend Architecture

8.1 Routing & Role Routing

React Router drives a single SPA with three top-level route trees, gated by a shared `ProtectedRoute` component that reads the decoded JWT's `role` claim from `AuthContext`:

```
<Routes>
  <Route path="/admin/*" element={<ProtectedRoute role="admin"><AdminLayout /></
ProtectedRoute>} />
  <Route path="/company/*" element={<ProtectedRoute role="company"><CompanyLayout /></
ProtectedRoute>} />
  <Route path="/candidate/*" element={<ProtectedRoute role="candidate"><CandidateLayout /
></ProtectedRoute>} />
  <Route path="/login" element={<Login />} />
</Routes>
```

`ProtectedRoute` redirects to `/login` if unauthenticated, and to a "not authorized" page if the token's role doesn't match the route tree — this is a UX convenience only; the actual authorization boundary is enforced server-side (Chapter 12), never trusted from the client alone.

8.2 Layouts

Each role has one layout component (`AdminLayout`, `CompanyLayout`, `CandidateLayout`) providing the persistent navigation shell (sidebar/topbar) for that panel, with `<Outlet />` rendering the active page. Layouts differ visually (accent color per role, per Chapter "UI Design") but share the same underlying `Card` / `Button` / `Modal` primitives from `components/common/`.

8.3 Component Architecture

- **`components/common/`** — unstyled-logic-light, purely presentational primitives reused across all three panels.
- **`components/charts/`** — Recharts wrappers, each accepting a plain data array/object as props and handling its own responsive container, tooltip, and color mapping — so page components never touch Recharts' raw API directly.
- **`components/interview/`** — the voice-interaction-specific components, isolated because they're the only components touching browser media APIs.
- **`components/report/`** — presentational components specific to rendering a completed `interview_reports` document, including the collapsible `ExplainabilitySection`.

8.4 Hooks

Hook	Purpose
`useInterviewSocket(sessionId)`	Opens/manages the WebSocket connection, exposes `status`, `currentQuestion`, `pushEvent`
`useAudioRecorder()`	Wraps `MediaRecorder`, exposes `start()`, `stop()`, `isRecording`, `audioBlob`
`useAuth()`	Reads/writes `AuthContext`, exposes `login()`, `logout()`, `role`, `companyId`
`useAnalyticsFilters()`	Manages dashboard filter state (time range, campaign) and syncs it to the URL query string for shareable/linkable dashboard views

8.5 Context & State Management

Global state is deliberately minimal — `AuthContext` (identity/role) and `ThemeContext` (role-based accent color) are the only React Contexts. All other state is local to the page/component that owns it, or lives server-side in MongoDB and is fetched on demand — there is no global client-side store (no Redux/Zustand) because the application's state is fundamentally server-owned; a client-side store would only introduce a second source of truth to keep in sync.

8.6 API Layer

`api/client.js` wraps `fetch`, attaching the JWT from `AuthContext` to every request and centralizing error handling (401 → redirect to login; 403 → "not authorized" toast). Each domain (`candidates.js`, `companies.js`, `interview.js`, `analytics.js`) exports plain async functions (`getReport(sessionId)`, `createCampaign(payload)`) — page components call these directly, no additional data-fetching abstraction layer, since the application's data-fetching needs are straightforward request/response, not requiring cache invalidation machinery.

8.7 Charts

Recharts components are configured with a shared color palette (imported from a single `chartColors.js` constant) so every chart across every panel uses identical semantic colors (green=strong, amber=attention, red=integrity concern), per the design principle in Chapter 16.

8.8 Animations

Framer Motion is scoped to exactly the interactions listed in Chapter 18 (UI Design) — score reveals, loading pulses, recording indicators, chart entrance — implemented as small `motion.div` wrappers around otherwise-static components, never as a page-wide animation framework.

8.9 Forms & Validation

Forms (campaign creation, candidate registration, company onboarding) use plain controlled React state with a lightweight validation utility (`utils/validation.js`) — field-level rules (required, min/max length, enum membership against the `allowed_*` lists fetched from the backend) run on submit and on blur; no heavyweight form library is introduced, since form complexity in this application is limited to a handful of straightforward, non-nested forms.

Chapter 9 — Backend Architecture

9.1 Layering

```
Routes (api/)  →  Services (core/, llm/, validation/, analytics/, reports/)  →
Repositories (db/)  →  MongoDB
```

- **Routes** handle HTTP/WebSocket concerns only: request parsing, RBAC dependency injection, response shaping. No business logic lives in a route handler.
- **Services** (Interview Manager and its submodules, LLM Service, Question Validator, Analytics Service, Report Generator) contain all business logic and are the only layer that composes multiple lower-level calls.
- **Repositories** (`db/mongo.py` collection accessors) are the only layer that issues raw MongoDB queries — services never construct a `motor` query directly, they call a named repository method (e.g., `sessions_repo.append_turn(session_id, turn)`), which keeps query shape changes isolated to one file per collection.

9.2 Dependency Injection

FastAPI's native `Depends()` mechanism is used throughout — `LLMService`, `SarvamClient`, and the Mongo database handle are constructed once at application startup and injected into route handlers and into the `InterviewManager` constructor, rather than instantiated ad hoc per request. This makes every service trivially mockable in tests (Chapter 21).

```
def get_llm_service() -> LLMService:
    return LLMService(primary=GroqProvider(), fallback=GeminiProvider())

def get_interview_manager(
    llm_service: LLMService = Depends(get_llm_service),
    sarvam: SarvamClient = Depends(get_sarvam_client),
    db=Depends(get_db),
) -> InterviewManager:
    return InterviewManager(llm_service, sarvam, db)
```

9.3 Authentication Middleware

A FastAPI dependency (`decode_jwt`) runs on every protected route, verifying signature and expiry and returning a `TokenPayload`; role-specific dependencies (`require_role("admin")`, etc., Chapter 12) compose on top of it.

9.4 Logging

Structured JSON logging (Python `logging` with a JSON formatter) at the service layer — every LLM Service call logs provider used, latency, retry count, and validation outcome (without logging full prompt/response content, to avoid duplicating PII already scoped out by the no-permanent-storage design); every Guardrail/Anti-Cheat trigger logs at `WARNING` level with session/company context for the Security Logs page (Chapter 4.1.3).

9.5 Configuration

All environment-specific values (API keys, MongoDB URI, recovery window duration, question-budget tuning constants) load through `config/settings.py` using Pydantic's `BaseSettings`, reading from environment variables — no configuration value is hardcoded inline in business logic, so every tunable constant referenced elsewhere in this document (e.g., `MAX_FOLLOW_UPS_PER_TOPIC`, the 5-minute recovery window) is in practice an env-var-backed setting with a sensible default.

9.6 Error Handling

A centralized exception handler (`middleware/error_handler.py`) maps internal exceptions to consistent HTTP responses:

Exception	HTTP Status	Response shape
`LLMServiceUnavailable` (both providers failed)	503	`{"error": "llm_unavailable", "retry_after": <seconds>}`
`ValidationError` (Pydantic)	422	`{"error": "validation_failed", "details": [...]}`
`PermissionError` (RBAC)	403	`{"error": "forbidden"}`
`SessionNotFoundError`	404	`{"error": "session_not_found"}`
`QuestionValidationExhausted` (Validator retries exhausted)	200 (fallback template served, not an error to the candidate)	falls back per FR-QV-10, logged internally as a warning

The candidate-facing interview flow specifically never surfaces a raw 5xx to the candidate mid-interview where a graceful fallback exists (e.g., LLM fallback provider, question template fallback) — errors that reach the candidate are limited to genuinely unrecoverable cases (both LLM providers down, Sarvam unavailable), presented as a plain "please try again shortly" state, not a stack trace.

Chapter 10 — Database Design (MongoDB)

10.1 Collection Overview

Collection	Purpose	Key Indexes
`users`	Admin/company account credentials and role	`{email: 1}` unique
`companies`	Tenant record + `allowed_*` configuration ceiling	`{_id: 1}`
`interview_campaigns`	Per-hiring-need interview configuration	`{company_id: 1, status: 1}`
`strategy_definitions`	Internal strategy metadata (Admin-managed, never candidate/company-visible)	`{_id: 1}`
`interview_mode_definitions`	Company-facing mode → internal parameter mapping	`{_id: 1}`
`candidates`	Candidate identity + structured resume profile	`{email: 1}`, `{campaign_id: 1}`
`interview_sessions`	Live/completed session, embedding `interview_state` and `turns[]`	`{candidate_id: 1}`, `{company_id: 1, created_at: -1}` (dashboard queries)
`interview_reports`	Final scored report + explainability	`{session_id: 1}` unique, `{company_id: 1, created_at: -1}`
`validator_logs`	Question Validator pass/fail history	`{session_id: 1, turn_number: 1}`

10.2 Full Field-Level Schemas

`users`

```
{ "_id": "ObjectId", "role": "admin | company", "email": "string", "password_hash": "string", "company_id": "ObjectId | null", "created_at": "ISODate" }
```

`companies`

```
{
  "_id": "ObjectId", "name": "string", "contact_email": "string",
  "allowed_languages": ["hi-IN", "en-IN"], "allowed_voices": ["bulbul-voice-1", "bulbul-
voice-3"],
  "allowed_strategies": ["strategy_a_token_driven", "strategy_b_conversational_bridge",
"hybrid"],
  "allowed_interview_modes": ["structured", "conversational", "balanced",
"deep_technical", "hr_behavioral", "campus_fresher"],
  "allowed_llm_tiers": ["groq_primary"], "max_campaigns": 20, "status": "active |
suspended",
  "branding": {"logo_url": "string | null", "accent_color": "#4f46e5"}, "created_at":
"ISODate"
}
```

`interview_campaigns`

```
{
  "_id": "ObjectId", "company_id": "ObjectId", "name": "string", "role_target": "string",
  "interview_type": "technical | hr | behavioral | mixed",
  "voice_config": {"voice_id": "string", "language": "hi-IN", "accent": "neutral |
regional"},
  "interview_mode": "balanced", "delegate_language_choice_to_candidate": true,
  "delegate_domain_choice_to_candidate": false, "allowed_candidate_languages": ["hi-IN",
"en-IN"],
  "status": "active | closed", "created_at": "ISODate"
}
```

Note: `min_questions` / `max_questions` are **not** stored as fixed campaign fields — per Dynamic Interview Planning, they are computed per-candidate at session start and stored on that candidate's `interview_sessions` document instead (see below).

`interview_mode_definitions`

```
{
  "_id": "balanced", "display_name": "Balanced Interview", "internal_strategy": "hybrid",
  "max_follow_ups_per_topic": 3, "topic_saturation_threshold": 0.75,
"completion_confidence_threshold": 0.75,
  "difficulty_policy": {"start": "medium", "progression": "standard"},
  "behavioral_templates_enabled": false, "is_default": true, "enabled": true
}
```

(One such document per mode: `structured`, `conversational`, `balanced`, `deep_technical`, `hr_behavioral`, `campus_fresher` — full parameter tables per mode are in Chapter 14.)

candidates

```
{
  "_id": "ObjectId", "campaign_id": "ObjectId", "company_id": "ObjectId",
  "name": "string", "email": "string", "experience_level": "string", "target_role":
"string",
  "resume_profile": {
    "candidate_name": "string", "target_role": "string",
    "experience": [{"title": "", "org": "", "description": "", "duration": ""}],
    "education": [{"degree": "", "institution": "", "year": ""}],
    "skills": ["string"], "projects": [{"name": "", "description": "", "technologies":
[]}],
    "certifications": ["string"], "strengths": ["string"], "technologies": ["string"],
"domains": ["string"],
    "potential_interview_topics": ["string"]
  },
  "created_at": "ISODate"
}
```

`interview_sessions`

```
{
  "_id": "ObjectId", "company_id": "ObjectId", "campaign_id": "ObjectId", "candidate_id":
"ObjectId",
  "language": "hi-IN", "interview_mode": "balanced", "status": "in_progress | completed",
  "question_budget": {"min_questions": 10, "max_questions": 16, "complexity_scores":
{"experience": 0.6, "skills": 0.7, "projects": 0.65}},
  "interview_state": {
    "current_question": "string", "current_topic": "string",
    "last_three_questions": ["string"], "last_three_answers": ["string"],
    "difficulty": "medium", "weak_areas": ["string"], "strong_areas": ["string"],
    "score_history": [0.78, 0.82], "topics_remaining": ["string"], "topics_covered":
["string"],
    "interview_blueprint": [{"topic_name": "string", "section": "string", "importance":
"mandatory|high|medium|low", "priority_rank": 1, "target_difficulty": "easy|medium|hard",
"estimated_coverage": "low|medium|high"}],
    "topic_coverage_map": {"Docker": {"status": "covered", "questions_asked": 3,
"follow_ups_asked": 2, "topic_confidence_score": 0.82, "last_score": 0.85}},
    "current_topic_follow_up_count": 0, "overall_coverage_percentage": 0.71,
"overall_interview_confidence": 0.68,
    "questions_asked_total": 9, "mandatory_topics_completed": true, "interview_phase":
"skills",
    "exploitation_attempt_count": 0
  },
  "turns": [
    {
      "turn_number": 1, "question": "string", "answer_transcript": "string",
"response_time_seconds": 42,
      "evaluation": {"technical_score": 0.8, "communication_score": 0.75,
"completeness_score": 0.7, "logical_flow_score": 0.85, "resume_consistency_score": 0.9,
"project_explanation_score": 0.6, "professionalism_score": 0.9, "response_quality_score":
0.75, "topic": "Docker", "readiness_score": 0.78, "suggests_follow_up": false,
"follow_up_reason": null},
      "was_follow_up": false, "was_blocked_by_guardrail": false,
"calibrate_hold_triggered": false, "cheating_risk_detected": false
    }
  ],
  "last_disconnected_at": "ISODate | null", "incomplete_coverage": false,
  "created_at": "ISODate", "completed_at": "ISODate | null"
}
```

`interview_reports`

```
{
  "_id": "ObjectId", "session_id": "ObjectId", "company_id": "ObjectId", "campaign_id":
"ObjectId",
  "overall_score": 78, "interview_readiness_score": 0.76,
  "resume_match_analysis": {"matched_skills": ["string"], "gap_skills": ["string"],
"consistency_notes": "string"},
  "topic_wise_scores": {"Python": 80, "React": 75}, "technical_skills_assessment":
"string", "communication_assessment": "string",
  "interview_risk_assessment": {"risks": ["string"], "severity": "low | medium | high"},
  "strengths": "string", "weaknesses": "string", "improvement_plan": "string",
"learning_resources": ["string"],
  "recruiter_summary": "string",
  "explainability": {
    "topic_selection_explanations": [{"topic": "string", "reason": "string"}],
    "difficulty_change_explanations": [{"turn": 4, "change": "easy → medium", "reason":
"string"}],
    "follow_up_explanations": [{"turn": 5, "reason": "string"}],
    "completion_explanation": {"reason": "string"},
    "readiness_score_explanation": {"formula_summary": "string", "score": 0.78}
  },
  "generated_at": "ISODate"
}
```

`validator_logs`

```
{ "_id": "ObjectId", "session_id": "ObjectId", "turn_number": 1, "attempt": 1, "passed":
false, "failed_rules": ["duplicate_question"], "candidate_question_text": "string",
"logged_at": "ISODate" }
```

10.3 Relationships (ER Summary)

```
companies (1) ——< interview_campaigns (many)
interview_campaigns (1) ——< candidates (many)
candidates (1) ——< interview_sessions (many, typically 1)
interview_sessions (1) —— interview_reports (1)
interview_sessions (1) ——< validator_logs (many)
```

All relationships are referenced by ObjectId, resolved application-side (no MongoDB `$lookup` joins in the hot interview-turn path — only used in analytics aggregation pipelines, where join cost is acceptable since those run on-demand for a dashboard view, not per interview turn).

10.4 Validation

Schema validation is enforced at the application layer via Pydantic models mirroring each collection shape (`db/models.py`), rather than MongoDB's native JSON Schema validators — chosen so that validation

logic, error messages, and the Python type system stay in one place, consistent with the rest of the FastAPI application's request/response validation.

Chapter 11 — API Documentation

11.1 Conventions

All endpoints return JSON. Authenticated endpoints require `Authorization: Bearer <JWT>`. Validation errors return HTTP 422 with a `details` array (Pydantic's native format). All list endpoints support `?limit=&offset=` pagination. Timestamps are ISO-8601 UTC.

11.2 Admin Endpoints

Method	Endpoint	Request Body	Response	Auth	Errors
POST	`/api/admin/companies`	`{name, contact_email, allowed_languages[], allowed_voices[], allowed_strategies[], allowed_interview_modes[], allowed_llm_tiers[], max_campaigns}`	`201 {company_id}`	admin	422 invalid payload
PATCH	`/api/admin/companies/{id}`	Partial of above	`200 {updated_fields}`	admin	404 not found
GET	`/api/admin/companies`	—	`200 [{...}]` (paginated)	admin	—
POST	`/api/admin/strategies`	`{id, display_name, description, prompt_template_ref, enabled}`	`201`	admin	409 duplicate id
PATCH	`/api/admin/strategies/{id}`	`{enabled}`	`200`	admin	404
POST	`/api/admin/interview-modes`	`{id, display_name, internal_strategy, max_follow_ups_per_topic, topic_saturation_threshold, completion_confidence_threshold, difficulty_policy, behavioral_templates_enabled}`	`201`	admin	422
GET	`/api/admin/analytics`	Query: `from`, `to`	`200 {platform metrics}`	admin	—

Method	Endpoint	Request Body	Response	Auth	Errors
GET	`/api/admin/security-logs`	Query: `company_id?`, `type?`, `from`, `to`	`200 [{...}]` (paginated)	admin	—
GET	`/api/admin/system-health`	—	`200 {llm_provider_status, sarvam_status, error_rate}`	admin	—

11.3 Company Endpoints

Method	Endpoint	Request Body	Response	Auth	Errors
POST	`/api/company/campaigns`	`{name, role_target, interview_type, voice_config, interview_mode, delegate_language_choice_to_candidate, delegate_domain_choice_to_candidate, allowed_candidate_languages}`	`201 {campaign_id, invite_link}`	company	422 (value outsic company's `allowed_*`), 4 (`max_campaign` exceeded)
GET	`/api/company/campaigns`	—	`200 [{...}]`	company	—
PATCH	`/api/company/campaigns/{id}`	Partial	`200`	company	404, 422
GET	`/api/company/campaigns/{id}/candidates`	—	`200 [{...}]` (paginated)	company	404
GET	`/api/company/reports/{session_id}`	—	`200 {full report incl. explainability}`	company	403 (not this company's session), 404
GET	`/api/company/reports/{session_id}/pdf`	—	`200` (binary, `application/pdf`)	company	403, 404
GET	`/api/company/analytics`	Query: `campaign_id?`, `from`, `to`	`200 {metrics}`	company	—
GET	`/api/company/analytics/export.csv`	Same query	`200` (binary, `text/csv`)	company	—
GET	`/api/company/analytics/export.pdf`	Same query	`200` (binary, `application/pdf`)	company	—

Method	Endpoint	Request Body	Response	Auth	Errors
PATCH	`/api/company/settings`	`{branding: {logo_url, accent_color}}`	`200`	company	422

11.4 Candidate Endpoints

Method	Endpoint	Request Body	Response	Auth	Errors
POST	`/api/candidates`	`{name, email, campaign_invite_token}`	`201 {candidate_id, jwt}`	none (invite-token-gated)	404 invalid/expired invite
POST	`/api/resume/upload`	multipart file	`200 {resume_profile}`	candidate	422 unsupported format, 502 LLM structuring failed after retries
POST	`/api/interview/start`	`{language?, domain?}` (only if delegated)	`201 {session_id}`	candidate	403 (choice outside campaign's delegated set)
GET	`/api/interview/{session_id}/next-question`	—	`200` (audio stream reference)	candidate	404
POST	`/api/interview/{session_id}/answer`	multipart audio + `response_time_seconds`	`200 {evaluation_summary?, next_question | completed}`	candidate	404, 503 (both LLM providers down)
GET	`/api/interview/{session_id}/report`	—	`200 {full report incl. explainability}`	candidate	403 (not own session), 404
WS	`/ws/interview/{session_id}?token=`	—	Server-push events (see 11.5)	candidate	4401 (invalid token, custom close code)

11.5 WebSocket Event Types

Event	Payload	When
`tts_ready`	`{audio_url}`	Question audio generated
`evaluating`	`{}`	Answer submitted, processing in progress
`next_question_ready`	`{question, audio_url}`	New question available
`session_recovered`	`{current_question}`	Reconnect within recovery window
`interview_completed`	`{session_id}`	Completion Engine fired

11.6 Status Code Summary

Code	Meaning in this system
200	Success
201	Resource created
403	RBAC/scope violation, or a requested value outside an `allowed_*` ceiling
404	Resource not found or not owned by the caller
422	Request validation failure
502	Upstream LLM/Sarvam failure that survived retries but is recoverable by re-attempting the same request
503	Both LLM providers unavailable — genuinely degraded service

Chapter 12 — Authentication

12.1 JWT Structure

```
class TokenPayload(BaseModel):
    sub: str                    # user id
    role: str                     # "admin" | "company" | "candidate"
    company_id: str | None
    campaign_id: str | None       # candidate tokens only
    exp: int
    iat: int
```

Access tokens are short-lived (recommended 30–60 minutes). A **refresh token** (long-lived, stored as an httpOnly cookie, opaque random string hashed in `users` / `candidates` collection) is used to mint new access tokens via `POST /api/auth/refresh`, without requiring re-login — standard practice to limit the exposure window of a leaked access token while keeping session-length interviews (which may run several minutes) from being interrupted by token expiry mid-interview.

12.2 RBAC Permission Matrix

Action	Admin	Company	Candidate
Manage companies			
Enable/disable strategies & modes			
Create campaigns		(own company)	
View own campaign's candidates			
View any candidate's report		(own company's only)	
View own report			
Take an interview			(own session only)
View platform-wide analytics			
View company analytics	(any)	(own only)	
View security logs			

12.3 Enforcement Implementation

```
def require_role(*allowed_roles: str):
    def dependency(token: TokenPayload = Depends(decode_jwt)):
        if token.role not in allowed_roles:
            raise HTTPException(403, "Insufficient permissions")
        return token
    return dependency

def require_own_company(token: TokenPayload = Depends(require_role("company")),
company_id: str = Path(...)):
    if token.company_id != company_id:
        raise HTTPException(403, "Cross-tenant access denied")
    return token
```

Every company- or candidate-scoped query filters by the `company_id` / `candidate_id` / `campaign_id` extracted from the verified token — **never** from a client-supplied request field — so a manipulated request body cannot widen access.

12.4 Session Recovery & Security

Session recovery (Chapter 6.15) requires the reconnecting WebSocket to present the **same** valid JWT the session was created under; a JWT for a different candidate ID cannot resume another candidate's session, even if the `session_id` is somehow known, because the Recovery Manager checks `session.candidate_id == token.sub` before restoring state.

12.5 Password & Credential Handling

Company and Admin passwords are hashed with a standard slow hash (bcrypt/argon2) before storage in `users`; candidates authenticate via campaign-invite-token-gated registration rather than a password, since candidates are typically one-time or infrequent users of a given campaign and a password-reset flow would add friction disproportionate to their usage pattern.

Chapter 13 — Interview Engine (Complete Detail)

13.1 Resume Processing Pipeline

```
Upload (PDF/DOCX, in-memory only)
  → Parser (PyMuPDF/pdfplumber/python-docx)
  → Cleaner (strip headers/footers/page numbers, normalize whitespace, repair line-wrap
breaks)
  → LLM Service.structure_resume() [Groq primary, Gemini fallback]
  → Structured Resume Profile (JSON, validated against schema)
  → MongoDB write (candidates.resume_profile)
  → Original file/buffer discarded (no disk or object-storage write ever occurs)
```

Why no permanent file storage: the platform's only downstream need from a resume is the structured profile; retaining the original file would add a data-retention/compliance surface with no corresponding functional benefit (Chapter 20, Security).

13.2 Dynamic Interview Planning — Complete Algorithm

13.2.1 Experience Analyzer

```
def analyze_experience(resume_profile: dict) -> float:
    years = estimate_total_years(resume_profile["experience"])
    role_seniority_bonus = 0.1 * count_senior_titles(resume_profile["experience"])
    internship_bonus = 0.05 * len([e for e in resume_profile["experience"] if
e.get("type") == "internship"])
    research_bonus = 0.1 if resume_profile.get("research") else 0
    raw_score = min(years / 8.0, 1.0) + role_seniority_bonus + internship_bonus +
research_bonus
    return min(raw_score, 1.0)   # normalized 0.0–1.0
```

13.2.2 Skill Complexity Analyzer

```
def analyze_skill_complexity(resume_profile: dict) -> float:
    primary_skills = resume_profile["skills"][:5]      # first 5 treated as primary
    secondary_skills = resume_profile["skills"][5:]
    primary_score = min(len(primary_skills) / 5.0, 1.0)
    secondary_score = min(len(secondary_skills) / 8.0, 1.0) * 0.5
    return min(primary_score + secondary_score, 1.0)
```

13.2.3 Project Complexity Analyzer

```
def analyze_project_complexity(resume_profile: dict) -> float:
    scores = []
    for project in resume_profile["projects"]:
        tech_count = len(project.get("technologies", []))
        description_depth = min(len(project.get("description", "").split()) / 40.0, 1.0)
        scores.append(min(tech_count / 5.0, 1.0) * 0.6 + description_depth * 0.4)
    return sum(scores) / max(len(scores), 1)
```

13.2.4 Question Budget Generator — Weighted Formula

```
complexity_index = (
    0.35 * experience_score +
    0.25 * skill_complexity_score +
    0.25 * project_complexity_score +
    0.15 * role_seniority_weight(target_role)
)

mode_multiplier = {
    "structured": 0.85, "conversational": 1.0, "balanced": 1.0,
    "deep_technical": 1.25, "hr_behavioral": 0.9, "campus_fresher": 0.8,
}[interview_mode]

raw_question_count = BASE_QUESTIONS + (complexity_index * SCALING_FACTOR *
mode_multiplier)
```

Illustrative bands (BASE_QUESTIONS=6, SCALING_FACTOR=19, sanity floor=6, ceiling=25):

Candidate profile	complexity_index	Resulting range
Fresher	~0.1–0.2	8–10
Junior	~0.3–0.4	10–14
Mid-level	~0.5–0.65	14–18
Senior	~0.75–0.95	18–25

```
def compute_question_budget(experience_score, skill_score, project_score, role, mode) ->
tuple[int, int]:
    complexity_index = (0.35*experience_score + 0.25*skill_score + 0.25*project_score +
0.15*role_seniority_weight(role))
    raw = BASE_QUESTIONS + complexity_index * SCALING_FACTOR * MODE_MULTIPLIER[mode]
    max_q = max(MIN_FLOOR, min(int(raw), MAX_CEILING))
    min_q = max(MIN_FLOOR, int(max_q * 0.6))       # min is a fraction of the computed
max, never fixed
    return min_q, max_q
```

Integration with the Completion Engine: the resulting `(min_questions, max_questions)` tuple is written to `interview_sessions.question_budget` at session start and is what the Completion Engine's

Maximum Question Guard (13.5) reads — replacing any platform-wide constant with a per-candidate value.

13.3 Blueprint Generation

One LLM Service call, informed by `resume_profile` + `question_budget` + `interview_type` / `role`, returns an ordered `list[TopicPlan]` (topic name, section, importance, priority rank, target difficulty, estimated coverage). The Topic Coverage Map is initialized in lockstep, one entry per blueprint topic, `status="not_started"`.

13.4 Topic Selection & Coverage Tracking

```
def select_next_topic(state) -> TopicPlan | None:
    candidates = [t for t in state.interview_blueprint if
state.topic_coverage_map[t.topic_name].status not in ("covered", "failed_abandoned")]
    if not candidates:
        return None
    importance_rank = {"mandatory": 0, "high": 1, "medium": 2, "low": 3}
    candidates.sort(key=lambda t: (importance_rank[t.importance], t.priority_rank))
    return candidates[0]

def update_coverage(state, topic_name, turn_score, was_follow_up):
    cov = state.topic_coverage_map[topic_name]
    cov.questions_asked += 1
    cov.follow_ups_asked += int(was_follow_up)
    cov.last_score = turn_score
    cov.topic_confidence_score = 0.6 * turn_score + 0.4 * cov.topic_confidence_score   #
recency-weighted
    if check_topic_saturation(cov):
        cov.status = "covered"
    elif check_failure_pivot(cov, last_two_scores(topic_name)):
        cov.status = "failed_abandoned"
    else:
        cov.status = "in_progress"
    recompute_aggregates(state)

def check_topic_saturation(cov) -> bool:
    return cov.questions_asked >= 2 and cov.topic_confidence_score >= 0.9

def check_failure_pivot(cov, last_two_scores) -> bool:
    return len(last_two_scores) == 2 and all(s < 0.3 for s in last_two_scores)
```

13.5 Follow-up Decision & Completion (Multi-Lock)

```
def needs_follow_up(state, evaluation, topic_plan, mode_config) -> bool:
    cov = state.topic_coverage_map[topic_plan.topic_name]
    if cov.follow_ups_asked >= mode_config.max_follow_ups_per_topic:
        return False
    if cov.status == "covered":
        return False
    if evaluation.readiness_score < 0.4:
        return cov.follow_ups_asked < 1
    if evaluation.suggests_follow_up and cov.topic_confidence_score <
mode_config.topic_saturation_threshold:
        return True
    return False

def should_complete_interview(state, mode_config) -> tuple[bool, str]:
    if state.questions_asked_total >= state.question_budget["max_questions"]:
        return True, "emergency_exit"
    if state.questions_asked_total < state.question_budget["min_questions"]:
        return False, "continue"
    if state.mandatory_topics_completed and state.overall_interview_confidence >=
mode_config.completion_confidence_threshold:
        return True, "clean_completion"
    return False, "continue"
```

13.6 Difficulty Control & Pacing

```
def update_difficulty(state, latest_score, response_time_seconds, transcript_word_count):
    high_cognitive_load = latest_score > 0.8 and response_time_seconds > 90
    if high_cognitive_load:
        return "CALIBRATE_HOLD"
    if latest_score >= 0.8:
        return increase(state.difficulty)
    elif latest_score >= 0.5:
        return state.difficulty
    else:
        return decrease(state.difficulty)
```

13.7 Evaluation & Readiness Score

Single LLM Service call per turn returns: technical, communication, completeness, logical flow, resume consistency, project explanation, professionalism, response quality scores, plus `suggests_follow_up` / `follow_up_reason`. Response time and topic coverage contribution are computed in Python. The Readiness Score is a fixed weighted formula (never asked of the LLM directly):

```
readiness_score = (
    0.25 * technical_score + 0.15 * communication_score + 0.15 * completeness_score +
    0.15 * resume_consistency_score + 0.10 * response_quality_score +
    0.10 * normalized_response_time_score + 0.10 * topic_coverage_score
)
```

13.8 Voice Processing Pipeline

```
Question text → Sarvam bulbul:v3 (TTS) → audio → WebSocket push → candidate listens
Candidate speaks → MediaRecorder capture → Submit → Sarvam saaras:v3 (STT) → transcript
```

No real-time interruption; evaluation begins only after explicit submission, keeping latency requirements relaxed (Chapter 20, Performance).

Chapter 14 — Interview Modes (Complete Reference)

14.1 Company-Facing Abstraction

```
Company → Interview Mode (business label only) → Interview Mode Manager → Strategy Selector → LLM Prompt Builder
```

Companies never see "Strategy A/B/Hybrid" — only mode names. The Interview Mode Manager resolves a mode to internal parameters via the `interview_mode_definitions` collection (Chapter 10.2), so adding a new mode requires no code deployment.

14.2 Mode-by-Mode Specification

14.2.1 Structured Interview

- **Purpose:** fast, direct, minimal conversation — campus drives, mass hiring.
- **Internal strategy:** Strategy A (token-driven).
- **Parameters:** `max_follow_ups_per_topic: 1`, `topic_saturation_threshold: 0.7`, minimal transition text, difficulty progression standard.
- **Example question style:** "What is the primary use case of Docker volumes?" (direct, no framing).

14.2.2 Conversational Interview

- **Purpose:** human-like discussion.
- **Internal strategy:** Strategy B (conversational bridging).
- **Parameters:** `max_follow_ups_per_topic: 3`, relaxed tone instructions, full bridge phrasing between topics.
- **Example:** "That's interesting — let's shift gears a bit and talk about how you approached the database layer."

14.2.3 Balanced Interview (Platform Default)

- **Purpose:** general hiring.
- **Internal strategy:** Hybrid.
- **Why default:** combines Strategy A's topic-drift protection with Strategy B's natural phrasing, giving the best trade-off for the widest range of hiring contexts without the extremes of either.
- **Parameters:** `max_follow_ups_per_topic: 3`, `topic_saturation_threshold: 0.75`, `completion_confidence_threshold: 0.75`.

14.2.4 Deep Technical Interview

- **Purpose:** senior engineers.
- **Internal strategy:** Hybrid, with elevated thresholds.
- **Parameters:** `max_follow_ups_per_topic: 5`, `topic_saturation_threshold: 0.9` (harder to mark "covered"), `completion_confidence_threshold: 0.85`, prompt bias toward "why," architecture, and design-tradeoff phrasing.
- **Example follow-up chain:** "Why FastAPI?" → "How did authentication work?" → "How did you handle horizontal scaling?"

14.2.5 HR & Behavioral Interview

- **Purpose:** soft-skill evaluation.
- **Internal strategy:** Hybrid, `behavioral_templates_enabled: true`.
- **Behavior:** the Prompt Builder automatically substitutes STAR-format templates (Situation, Task, Action, Result) for blueprint sections tagged `behavioral` — covering leadership, conflict resolution, communication, decision-making, teamwork — regardless of the candidate's technical resume content.

14.2.6 Campus/Fresher Interview

- **Purpose:** entry-level candidates.
- **Internal strategy:** Strategy A or Hybrid (configurable), lower starting difficulty.
- **Parameters:** `difficulty_policy.start: "easy"`, `max_follow_ups_per_topic: 2`, prompt instructions favor encouraging phrasing and resume-anchored (not open-ended) question framing.

14.3 Interview Mode Manager — Class

```
class InterviewModeManager:
    def __init__(self, db):
        self.db = db  # reads interview_mode_definitions

    def resolve_mode(self, mode_id: str) -> ModeConfig:
        doc = self.db.interview_mode_definitions.find_one({"_id": mode_id, "enabled":
True})
        if not doc:
            raise ValueError(f"Interview mode '{mode_id}' not found or disabled")
        return ModeConfig(**doc)
```

14.4 Flow Diagram

```
Company selects Interview Mode (campaign creation)
        |
        ▼
InterviewModeManager.resolve_mode(mode_id) → ModeConfig
        |
        ▼
ModeConfig feeds: Strategy Selector (internal_strategy), Difficulty Controller
(difficulty_policy),
                  Follow-up Engine (max_follow_ups_per_topic), Completion Engine
(thresholds),
                  Prompt Builder (behavioral_templates_enabled)
```

Chapter 15 — Question Validation (Complete Module)

15.1 Pipeline Position

```
Strategy Engine → LLM (question generated) → Question Validator → [pass] → TTS → Candidate
                                                                    |
                                                                [fail] → regenerate (bounded retries) → fallback template if exhausted
```

15.2 Validation Rules

Rule	Method	Threshold/Logic
Duplicate detection	Keyword overlap (Jaccard similarity on tokenized question) + embedding cosine similarity against all prior questions in the session	Reject if keyword overlap > 0.6 OR embedding similarity > 0.85
Topic relevance	Embedding similarity between the question and the current `TopicPlan.topic_name` + its blueprint description	Reject if similarity < 0.4
Difficulty alignment	Heuristic scoring (question length, presence of "why/how/design" vs. "what is") cross-checked against `Difficulty Controller`'s current tier	Reject on strong mismatch (e.g., a one-clause factual question tagged "hard")
Resume/ blueprint alignment	Keyword check — every named skill/project/technology in the question text must appear in `resume_profile` or `interview_blueprint`	Reject if a referenced entity is absent (prevents hallucinated references)
Follow-up validity	If `was_follow_up=True`, question text must share at least one keyword/entity with the immediately prior answer transcript	Reject if no overlap (indicates a disguised topic change)
Clarity & length	Word count between 6 and 40; ends in `?`; no unresolved template placeholders	Reject outside bounds or on placeholder leakage
Interview Mode compliance	Structured mode: reject multi-clause/conversational-framing questions; Conversational mode: reject overly terse questions lacking any transition phrase when a topic change is occurring	Mode-specific heuristic

15.3 Duplicate Detection — Algorithm Detail

```
def is_duplicate(candidate_question: str, prior_questions: list[str], embed_fn) -> bool:
    q_tokens = set(tokenize(candidate_question.lower()))
    for prior in prior_questions:
        p_tokens = set(tokenize(prior.lower()))
        jaccard = len(q_tokens & p_tokens) / len(q_tokens | p_tokens)
        if jaccard > 0.6:
            return True
    candidate_vec = embed_fn(candidate_question)
    for prior in prior_questions:
        if cosine_similarity(candidate_vec, embed_fn(prior)) > 0.85:
            return True
    return False
```

The embedding model used here is a small, local sentence-embedding model (not an LLM call) — chosen specifically because the Validator must remain deterministic and low-latency; calling an LLM to check duplication would violate the "not another LLM" requirement and double per-turn latency.

15.4 Regeneration Policy

```
MAX_VALIDATION_RETRIES = 2

async def get_validated_question(context) -> str:
    for attempt in range(MAX_VALIDATION_RETRIES + 1):
        question = await llm_service.evaluate_and_generate(context)
        result = validator.validate(question, context)
        log_validation_attempt(context.session_id, context.turn_number, attempt, result)
        if result.passed:
            return question
    return fallback_template_for(context.current_topic, context.difficulty)
```

15.5 Logging & Analytics

Every attempt (pass or fail, with failed-rule list) is written to `validator_logs` (Chapter 10.2) — this feeds the "Strategy Effectiveness" and general quality metrics in Company/Admin Analytics (Chapter 16), letting the platform operator see, e.g., which Interview Mode has the highest validator failure rate and warrants prompt-template tuning.

15.6 `question_validator.py` — Function Reference

```
class QuestionValidator:
    def validate(self, question: str, context: ValidationContext) -> ValidationResult:
        failed_rules = []
        if self._is_duplicate(question, context.prior_questions):
failed_rules.append("duplicate_question")
        if not self._is_topic_relevant(question, context.topic_plan):
failed_rules.append("topic_relevance")
        if not self._difficulty_aligned(question, context.difficulty):
failed_rules.append("difficulty_alignment")
        if not self._resume_aligned(question, context.resume_profile, context.blueprint):
failed_rules.append("resume_alignment")
        if context.was_follow_up and not self._valid_follow_up(question,
context.last_answer): failed_rules.append("follow_up_validity")
        if not self._clear_and_well_formed(question):
failed_rules.append("clarity_length")
        if not self._mode_compliant(question, context.mode_config):
failed_rules.append("mode_compliance")
        return ValidationResult(passed=len(failed_rules) == 0, failed_rules=failed_rules)
```

Each `_is_*` / `_valid_*` method is a small, independently testable pure function (Chapter 21, Unit Testing).

Chapter 16 — Analytics (Complete Dashboard)

16.1 Scope

Two dashboards share the same underlying Analytics Service, differing only in scope: **Company Analytics** (filtered to `company_id`, optionally further to `campaign_id`) and **Admin Analytics** (platform-wide, no `company_id` filter, with an additional Security Logs cross-section).

16.2 Metrics (Complete List)

Metric	Definition	Aggregation source
Most common weak skills	Topics most frequently landing in a candidate's `weak_areas`	`interview_sessions.interview_state.weak_areas`, grouped
Strongest skills	Topics most frequently in `strong_areas`	same, `strong_areas`
Average interview score	Mean `overall_score`	`interview_reports`
Average interview duration	Mean `completed_at - created_at`	`interview_sessions`
Average confidence (Readiness Score)	Mean `interview_readiness_score`	`interview_reports`
Difficulty distribution	Count of turns per difficulty tier	`interview_sessions.turns[].evaluation` cross-referenced with the difficulty at that turn
Interview completion rate	`completed` sessions / total sessions	`interview_sessions.status`
Drop-off rate	Sessions abandoned (never reconnected past the recovery window) / total	`interview_sessions` where `status="in_progress"` and `last_disconnected_at` is stale
Question count distribution	Histogram of `questions_asked_total` at completion	`interview_sessions.interview_state`
Strategy effectiveness	Completion rate & average score, grouped by resolved internal strategy	`interview_sessions.interview_mode` joined to `interview_mode_definitions.internal_strategy`

Metric	Definition	Aggregation source
Interview Mode effectiveness	Completion rate & average score, grouped by `interview_mode`	`interview_sessions`
Average follow-up count	Mean `follow_ups_asked` across all topics	`interview_sessions.interview_state.topic_coverage_map`
Average topic coverage	Mean `overall_coverage_percentage`	`interview_sessions.interview_state`
Cheating incidents	Count of turns with `cheating_risk_detected=true`	`interview_sessions.turns[]`
Guardrail triggers	Count of turns with `was_blocked_by_guardrail=true`	`interview_sessions.turns[]`
Resume consistency statistics	Distribution of `resume_match_analysis.gap_skills` counts	`interview_reports`
Top failed topics	Topics most frequently `failed_abandoned`	`interview_sessions.interview_state.topic_coverage_map`
Most difficult topics	Topics with lowest average `topic_confidence_score`	same
Most successful Interview Mode	Highest completion rate × average score composite	`interview_sessions` + `interview_reports`

16.3 Aggregation Pipeline Example

```
pipeline = [
    {"$match": {"company_id": company_id, "created_at": {"$gte": from_date, "$lte":
to_date}}},
    {"$group": {
        "_id": "$interview_mode",
        "avg_score": {"$avg": "$overall_score"},
        "count": {"$sum": 1},
        "completion_rate": {"$avg": {"$cond": [{"$eq": ["$status", "completed"]}, 1,
0]}},
    }},
    {"$sort": {"avg_score": -1}},
]
```

Every dashboard metric follows this shape: `$match` scope + time range → `$group` by the relevant dimension → `$sort` / `$project` for presentation — all executed on demand per dashboard load or filter change, not materialized into a separate collection, since MongoDB's aggregation performance at this data

scale (per-company session counts) does not require pre-computation, and on-demand computation guarantees the dashboard is never stale.

16.4 MongoDB Collections Used

Entirely reuses `interview_sessions`, `interview_reports`, and `validator_logs` — no dedicated analytics collection exists, per the "no duplicate functionality" design constraint carried through every architecture revision.

16.5 Dashboard Widgets → Metric Mapping

See Chapter "UI Design" (companion Final Platform Document) for the visual layout; each widget maps 1:1 to one row of the table in §16.2, computed via its own aggregation call, allowing widgets to load and error independently (a slow "Strategy Effectiveness" aggregation never blocks the "Average Score" counter from rendering).

16.6 Filtering & Time Ranges

All dashboard endpoints accept `from` / `to` ISO date query parameters and an optional `campaign_id` (Company Panel only). The frontend's `useAnalyticsFilters` hook (Chapter 8.4) persists the selected range to the URL, making dashboard views shareable/bookmarkable.

16.7 Export

- **CSV:** aggregation result serialized directly via `pandas.DataFrame.to_csv()`.
- **PDF:** same WeasyPrint pipeline used for recruiter reports (Chapter 17.4), with a dashboard-shaped Jinja2 template rather than a single-candidate template.

Chapter 17 — Report Generation (Complete)

17.1 Two Output Forms

Form	Technology	Rationale
In-app report	React components	Fast, interactive, linkable, the primary read for both candidates and recruiters
Downloadable PDF	WeasyPrint (HTML/CSS → PDF), Jinja2 templating	Chosen over ReportLab specifically because the report's visual structure (score rings, colored bars, section cards) is far more naturally expressed in CSS than in imperative canvas drawing

17.2 Report Assembly Pipeline

```
Session completed
  → Precompute aggregates in Python (per-topic averages, difficulty progression, score trend)
  → LLM Service.check_resume_consistency() → feeds Interview Risk Assessment
  → LLM Service.generate_final_report(aggregates, interview_state, resume_profile) → qualitative sections
  → ExplainabilityGenerator.generate(session) → explainability object (Chapter 18)
  → Assemble interview_reports document
  → MongoDB insert
```

17.3 Report Contents

Overall Score, Interview Readiness Score, Resume Match Analysis, Topic-wise Scores, Technical Skills Assessment, Communication Assessment, Interview Risk Assessment, Strengths, Weaknesses, Personalized Improvement Plan, Learning Resources, Recruiter-style Summary, and the collapsible Explainability section (placed last, per the design principle that recruiters scan scores first).

17.4 PDF Generation Pipeline

```
interview_reports document + company branding (logo, accent color from companies.branding)
  → Jinja2 render (report_template.html)
  → WeasyPrint HTML→PDF
  → Returned on-demand via GET /api/.../report/pdf (never stored — generated fresh per request, consistent with the platform's no-unnecessary-persistence principle)
```

17.5 Scoring Precomputation Discipline

All numeric aggregation (averages, trends, percentages) is computed in Python before the final LLM call — the LLM is asked only to write qualitative prose *about* numbers it is given, never to calculate them, since LLM arithmetic is unreliable and unauditable compared to a deterministic formula.

Chapter 18 — Explainability Module (Complete)

18.1 Design Principle

The Explainability Module narrates decisions that are **already deterministic and already logged** elsewhere in the system (Difficulty Controller, Topic Priority Manager, Completion Engine, the Readiness Score formula). It computes nothing new — it is a read-only presentation layer over existing state, which is precisely why it required no architecture change to add.

18.2 Generation Pipeline

```
Interview completed
  → Read interview_sessions.turns[] and interview_state (no new writes needed to gather
this)
  → ExplainabilityGenerator (deterministic Python, NOT an LLM call):
       Topic Selection Explainer   → reads TopicPlan.importance
       Difficulty Explainer         → reads score_history + difficulty transition points
       Follow-up Explainer          → reads evaluation.suggests_follow_up +
follow_up_reason (already stored)
       Completion Explainer         → reads Completion Engine's stored trigger reason
       Readiness Score Explainer    → reads the same weighted formula already used to
compute the score
  → interview_reports.explainability
```

18.3 Why Deterministic, Not LLM-Generated

Asking an LLM to "explain" a decision risks it inventing a plausible-sounding but inaccurate reason. Template-filling the actual trigger conditions — which are already structured data — is both cheaper (no extra LLM call) and more trustworthy, since the explanation is guaranteed to match the actual mechanism that fired.

18.4 Class Reference

```
class ExplainabilityGenerator:
    def generate(self, session: dict) -> dict:
        return {
            "topic_selection_explanations": self._explain_topic_selections(session),
            "difficulty_change_explanations": self._explain_difficulty_changes(session),
            "follow_up_explanations": self._explain_follow_ups(session),
            "completion_explanation": self._explain_completion(session),
            "readiness_score_explanation": self._explain_readiness_score(session),
        }
```

Full method bodies are template-string generators over stored fields (see companion "Final Platform Document" for worked examples matching the platform's own terminology, e.g., "Docker was asked because it was marked as a primary skill in your resume.").

18.5 Storage & API

Nested field on the existing `interview_reports` document — no new collection, no new endpoint; served through the existing `GET /.../report` response.

18.6 UI Presentation

Collapsible section, placed last in both the in-app and PDF report, following the principle that scores are the primary read and explanations are available on demand for audit/trust purposes.

Chapter 19 — Technology Stack (With Alternatives Rejected)

Layer	Chosen	Alternatives considered	Why chosen	Trade-off accepted
Frontend framework	React	Vue, Svelte	Largest ecosystem, team familiarity, strong charting/ animation library support	Larger bundle than Svelte — acceptable for an internal/B2B tool, not a marketing site
Styling	Tailwind CSS	CSS Modules, styled-components	Fast iteration, consistent design tokens across 3 panels without a separate design system	Verbose class lists in JSX — mitigated by componentizing repeated patterns
Charts	Recharts	Chart.js, D3 (raw), Victory	Declarative React component API, composes naturally with component state	Less flexible for highly custom/novel chart types than raw D3 — not needed here
Animation	Framer Motion	CSS animations, React Spring	State-driven animation (animate on filter change, not just mount)	Adds one dependency — scoped narrowly, not used pervasively
Backend framework	FastAPI	Django, Flask	Native async support (critical for concurrent LLM/ STT calls), automatic OpenAPI docs, Pydantic-native validation	Smaller batteries-included ecosystem than Django — not needed, this app doesn't need Django's admin/ORM
Database	MongoDB	PostgreSQL, PostgreSQL+pgvector	Document shape matches nested session/report data naturally; no joins needed in the hot interview-turn path	Less rigid schema enforcement than Postgres — mitigated via Pydantic-layer validation
Vector DB / RAG	**None**	Qdrant, pgvector	Resume profile is small (a few hundred tokens) and single-purpose; direct injection into the prompt is simpler and equally effective at this scale	Would need to reconsider if job-description matching or a shared question bank were added later (Chapter 23)
Cache	**None (no Redis)**	Redis	Session recovery and live state reads use MongoDB directly since `interview_state` is already a complete, bounded snapshot	Slightly higher per-read latency than an in-memory cache — not significant at current session-concurrency scale

Layer	Chosen	Alternatives considered	Why chosen	Trade-off accepted
File storage	**None (no S3/AWS)**	S3, GCS	Nothing needs to persist beyond the structured profile and transcripts; removing this layer removes an entire compliance/infra surface	Cannot later add raw-audio behavioral analysis without reintroducing storage narrowly for audio (Chapter 23)
Primary LLM	Groq — `llama-3.3-70b-versatile`	OpenAI GPT-4o, Anthropic Claude	Low-latency inference matters directly since evaluation sits in the live interview's critical path	Slightly lower ceiling on the most complex reasoning tasks vs. frontier closed models — mitigated by the deterministic guardrails around every LLM call, which reduce reliance on raw model judgment
Fallback LLM	Gemini 2.5 Flash	A second Groq-hosted model	An independent provider gives genuine redundancy against a Groq-specific outage, not just a different model on the same infrastructure	One more provider integration to maintain — isolated entirely within the LLM Service Layer
STT	Sarvam AI `saaras:v3`	Whisper, Google STT	Best-in-class Indian-language transcription including code-mixing (Hinglish, etc.)	Narrower language coverage outside the Indian-language set than Google STT — acceptable given target market
TTS	Sarvam AI `bulbul:v3`	ElevenLabs, Google TTS	Natural multilingual voice matched to the same provider family as STT, low streaming latency	Fewer voice options than ElevenLabs — acceptable for a professional-interview tone, not needing character voices
PDF generation	WeasyPrint	ReportLab	HTML/CSS-driven layout suits visually structured reports; ReportLab better suited to simple text-heavy documents, not used here	Slightly heavier runtime dependency (a headless-browser-adjacent rendering engine) than pure ReportLab
Auth	JWT	Session cookies + server-side session store	Stateless, scales horizontally with FastAPI without sticky sessions	Requires refresh-token machinery for long-lived sessions — implemented (Chapter 12.1)

Chapter 20 — Non-Functional Requirements

20.1 Performance

Per-turn round trip (STT → evaluation/next-question LLM call → validation → TTS) targets completion within a few seconds under normal provider latency — acceptable given the deliberate no-real-time-streaming design (Chapter 13.8). Blueprint generation and question-budget computation (one-time, at session start) may take longer and shall present a setup/loading state rather than a blocked UI.

20.2 Scalability

FastAPI is stateless per request (all durable state in MongoDB), enabling horizontal scaling behind a load balancer with no session affinity requirement. Strategies and Interview Modes are configuration data, not code, so new ones are added without a deployment (Chapter 14.3).

20.3 Reliability & Availability

LLM calls automatically fail over from Groq to Gemini on error (Chapter 6.6). Sessions survive transient WebSocket disconnects via the Recovery Manager (Chapter 6.15) without data loss or re-invoking the LLM.

20.4 Maintainability

All provider-specific code is isolated to the LLM Service Layer's provider modules (Chapter 9.1 layering). All deterministic decision logic (difficulty, completion, validation, coverage) remains in small, independently testable pure Python functions, separated from LLM-dependent code paths — directly enabling the unit-testing strategy in Chapter 21.2.

20.5 Security

RBAC is enforced server-side on every endpoint (Chapter 12.3); no permanent storage of raw resumes or audio reduces data-retention risk by design (Chapter 13.1); Guardrail and Anti-Cheat checks run on every submitted transcript without exception (Chapter 6.13–6.14).

20.6 Accessibility

The candidate-facing interview screen provides a text fallback of the current question alongside audio (not audio-only), and all interactive controls (Start Answering, Submit) are standard, keyboard-and-screen-

reader-navigable HTML controls rather than custom-drawn canvas elements, per standard WCAG-adjacent practice for a voice-first but not voice-exclusive interface.

20.7 Usability

Company users are never exposed to implementation-level terminology (Chapter 14.1); animation is scoped to state-change communication only, not decoration (companion Final Platform Document, UI chapter).

20.8 Localization

Voice interaction supports the languages enabled in `companies.allowed_languages` (an Admin-managed, extensible list); UI text localization (translating panel labels themselves) is not in the current scope but the architecture does not preclude adding an i18n layer to the React frontend later, since no component hardcodes English strings inline outside of clearly isolated content areas.

Chapter 21 — Testing Strategy

21.1 Unit Testing

Every deterministic module (Difficulty Controller, Coverage Manager, Follow-up Decision Engine, Completion Engine, Question Validator's individual `_is_*` rule methods, Dynamic Interview Planner's analyzers, Explainability Generator's template methods) is unit-tested with plain input/output assertions — no mocking of external services required, since these modules by design never call an LLM or external API. This is the direct payoff of the architecture's deterministic/LLM-boundary discipline (Chapter 20.4): the majority of the system's decision logic is testable without any network or LLM cost.

```
def test_difficulty_increases_after_high_score():
    result = update_difficulty(state=mock_state(difficulty="medium"), latest_score=0.85,
response_time_seconds=30, transcript_word_count=60)
    assert result == "hard"

def test_calibrate_hold_on_high_score_long_response():
    result = update_difficulty(state=mock_state(difficulty="medium"), latest_score=0.85,
response_time_seconds=120, transcript_word_count=60)
    assert result == "CALIBRATE_HOLD"

def test_completion_emergency_exit_at_max_questions():
    triggered, reason = should_complete_interview(mock_state(questions_asked_total=25,
question_budget_max=20), mock_mode_config())
    assert triggered and reason == "emergency_exit"
```

21.2 Integration Testing

LLM Service tests use a recorded-response fixture layer (stub provider returning canned, schema-valid JSON) to verify retry/fallback logic without live API calls: force the primary provider stub to fail, assert the fallback provider stub is invoked, assert final response validation.

21.3 API Testing

FastAPI's `TestClient` drives full request/response cycles against a test MongoDB instance (via `mongomock` or a disposable Docker container), covering: RBAC enforcement (a company token cannot read another company's report → expect 403), validation error shapes, and the full happy-path interview flow end-to-end with LLM/Sarvam calls stubbed.

21.4 UI Testing

Component-level tests (React Testing Library) for interactive components (`AudioRecorder`, `QuestionCard`, dashboard filter interactions); a small set of end-to-end tests (Playwright) covering the critical path: register → upload resume → complete a stubbed-backend interview → view report.

21.5 Load Testing

Locust or k6 scripts simulating concurrent interview sessions against a staging environment with LLM/Sarvam calls pointed at low-cost/sandboxed provider tiers, to validate the stateless-FastAPI horizontal-scaling assumption (Chapter 20.2) under realistic concurrency.

21.6 Security Testing

Guardrail Router regression suite: a maintained list of known prompt-injection phrasings run through `check_guardrail()` to confirm detection; RBAC boundary tests (Chapter 21.3) doubling as a basic access-control security test; dependency vulnerability scanning (`pip-audit`, `npm audit`) as part of CI.

21.7 Acceptance Testing

Scenario-based acceptance tests mapped directly to SRS functional requirements (`FR-*` IDs) — e.g., FR-COMP-2 ("shall never terminate below the dynamically computed minimum question count") is verified by an acceptance test that runs a full mocked interview scripted to attempt early completion and asserts the session continues past `min_questions`.

Chapter 22 — Deployment

22.1 Containerization

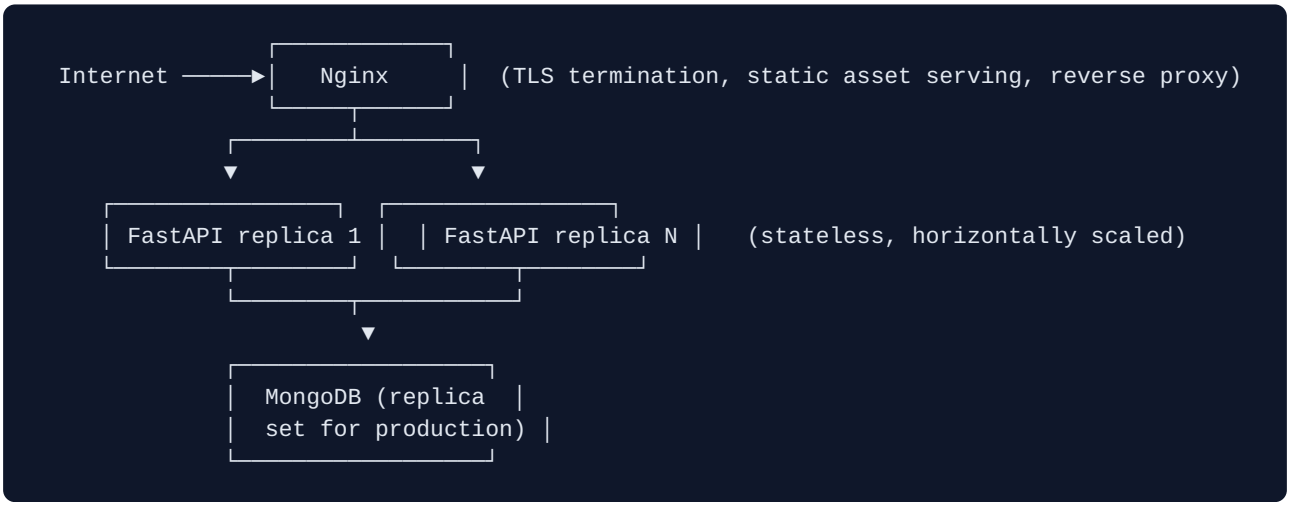
```
# docker-compose.yml (development)
services:
  backend:
    build: ./backend
    env_file: .env
    ports: ["8000:8000"]
    depends_on: [mongo]
  frontend:
    build: ./frontend
    ports: ["3000:3000"]
    depends_on: [backend]
  mongo:
    image: mongo:7
    volumes: ["mongo_data:/data/db"]
    ports: ["27017:27017"]
volumes:
  mongo_data:
```

22.2 Environment Variables

Variable	Purpose
`MONGO_URI`	MongoDB connection string
`GROQ_API_KEY`	Primary LLM provider credential
`GEMINI_API_KEY`	Fallback LLM provider credential
`SARVAM_API_KEY`	STT/TTS provider credential
`JWT_SECRET`	Access/refresh token signing key
`JWT_ACCESS_TTL_MINUTES`	Access token lifetime
`SESSION_RECOVERY_WINDOW_SECONDS`	WebSocket reconnect window (default 300)
`MAX_QUESTION_CEILING`	Absolute sanity ceiling for the Dynamic Question Budget
`CORS_ALLOWED_ORIGINS`	Frontend origin(s) permitted to call the API

All loaded via `config/settings.py` (Pydantic `BaseSettings`) — no secret is ever hardcoded in source.

22.3 Production Architecture



22.4 Nginx Responsibilities

TLS termination, gzip compression, static React build serving, `/api/*` and `/ws/*` reverse-proxying to the FastAPI backend pool (with WebSocket upgrade headers explicitly forwarded, since WebSocket proxying requires `Upgrade` / `Connection` header passthrough that Nginx does not enable by default).

22.5 CI/CD

```
On push to main:
  1. Lint (ruff/eslint) + type-check (mypy/tsc)
  2. Run unit + integration test suites (Chapter 21.1–21.2)
  3. Build Docker images (backend, frontend)
  4. Push images to registry
  5. Deploy to staging → run API + acceptance test suite against staging
  6. Manual approval gate → deploy to production
```

22.6 Monitoring & Logging

Structured JSON logs (Chapter 9.4) shipped to a log aggregator (e.g., a hosted logging service); FastAPI request latency and error-rate metrics exported for the Admin Panel's System Health page (Chapter 4.1.3), alongside explicit LLM-provider health checks (periodic lightweight ping to Groq/Gemini, surfaced as `system_health.llm_provider_status`).

22.7 Backups

MongoDB replica-set-native backup/point-in-time-recovery tooling scheduled on a standard interval; since no resume files or audio are persisted (Chapter 13.1), backup scope is limited to structured, already-minimized data, keeping backup size and restore complexity low relative to a platform that also stores raw media.

Chapter 23 — Future Enhancements (Realistic Scope Only)

Per explicit project constraint, no video analysis, facial expression detection, eye tracking, or emotion detection is planned — these are deliberately excluded as they would contradict the platform's existing "measurable behavioral signals, not inferred emotion" design principle (SRS §7.2).

Realistic, architecturally-consistent future work:

- **Interview Replay**: a turn-by-turn playback view (audio + transcript + evaluation, synchronized) for recruiters — purely additive to the existing report view, no new backend computation, since all underlying data is already stored.
- **Transcript-level RAG**: if interviews grow long enough (15+ turns) that a fixed 3-turn recency window starts losing relevant earlier context, a second retrieval collection (`interview_chunks`) could be added — the architecture's existing "no RAG needed at current scale" decision (Chapter 19) already documents exactly this as the reconsideration trigger.
- **Job-description matching**: if campaigns need to score candidates against a specific job description (not just their own resume), a small vector index over job descriptions (not resumes) would be the natural home for a first reintroduction of a vector database.
- **Redis caching**: if concurrent session volume grows enough that per-turn MongoDB reads become a bottleneck, an in-memory cache for active `interview_state` is a clean, additive change, since `interview_state` is already a self-contained object.
- **Localized UI strings**: extending the existing React component structure with an i18n library for full panel-language localization, not just interview-voice language.
- **Per-company LLM tier selection**: `companies.allowed_llm_tiers` already exists as a field (Chapter 10.2); a future premium tier could permit a stronger model as primary rather than fallback, with zero architecture change, only a new enum value.

Chapter 24 — Appendices

24.1 Glossary

See SRS §1.3 for the authoritative term list (STT/TTS, LLM, RBAC, Turn, Blueprint, Interview State, Interview Mode, Strategy, Readiness Score, Guardrail) — repeated here for convenience:

Term	Meaning
Turn	One question–answer exchange within a session
Blueprint	The ordered, weighted topic list generated from the resume
Interview State	The live object tracking everything about an in-progress interview
Interview Mode	Company-facing label mapping to an internal strategy + rule set
Strategy (A/B/Hybrid)	Internal, non-company-facing prompt-construction techniques
Readiness Score	The platform's deterministic performance/behavioral estimate
Guardrail	Deterministic exploit/prompt-injection filter

24.2 Abbreviations

STT, TTS, LLM, RBAC, JWT, SRS, SDD, FR (Functional Requirement), NFR (Non-Functional Requirement), KPI, ER (Entity-Relationship), CI/CD, TLS.

24.3 Complete API Table

See Chapter 11 for the full, authoritative endpoint reference.

24.4 Complete Folder Tree

See Chapter 7 for the full, authoritative project structure.

24.5 Complete Database Schema

See Chapter 10 for the full, authoritative collection field reference.

24.6 Complete Project Flow

See Chapter 5 for the full, authoritative end-to-end sequence.

24.7 Coding Standards

Area	Standard
Python naming	`snake_case` for functions/variables, `PascalCase` for classes, module-per-responsibility (Chapter 7)
React naming	`PascalCase` component files, `camelCase` hooks prefixed `use`
API design	Nouns for resources, HTTP verbs for actions, consistent `{error, details}` error shape (Chapter 9.6)
Error handling	Centralized exception handler (Chapter 9.6); no bare `except:` blocks; every external call (LLM, Sarvam) wrapped with a typed exception
Logging	Structured JSON, no PII/prompt-content logging beyond what's already persisted in MongoDB by design
Configuration	Environment-variable-backed `Settings` object (Chapter 22.2); no hardcoded secrets or tunable constants inline
Dependency Injection	FastAPI `Depends()` for all services (Chapter 9.2); constructors accept dependencies, never instantiate them internally
Repository Pattern	All MongoDB access through named repository functions (Chapter 9.1); no raw `motor` queries inside service/business logic
Service Layer	All business logic in `core/`, `llm/`, `validation/`, `analytics/`, `reports/`; routes are thin (Chapter 9.1)